Apache Mahout QDP: GPU-Accelerated Quantum Data Plane

Guan-Ming Chiu*
National Taiwan University
Taipei, Taiwan
guanmingchiu@apache.org

Kuan-Hao Huang*
National Taichung University of Education
Taichung, Taiwan
richhuang@apache.org

Hsien-Cheng Huang*
National Yang Ming Chiao Tung University
Hsinchu, Taiwan
hcr@apache.org

Jie-Kai Chang*
Yuan Ze University
Taipei, Taiwan
jiekaichang@apache.org

ABSTRACT

Quantum machine learning (QML) pipelines require encoding classical data into quantum states before any computation can proceed. Existing frameworks such as PennyLane and Qiskit construct encoding circuits on the CPU, one vector at a time, creating a severe data loading bottleneck that dominates end-to-end training time. We demonstrate **Apache Mahout QDP** (Quantum Data Plane), a GPU-accelerated data pipeline engine built with Rust and CUDA. QDP provides a complete pipeline from columnar storage formats (Parquet, Arrow) through normalization and quantum state encoding to zero-copy tensor output via DLPack, achieving orders-of-magnitude speedups over existing frameworks. Our interactive demonstration allows users to (1) explore the end-to-end encoding pipeline with configurable parameters, (2) compare latency and throughput against PennyLane and Qiskit in real time, and (3) analyze performance scaling across qubit counts.

PVLDB Reference Format:
Guan-Ming Chiu, Kuan-Hao Huang, Hsien-Cheng Huang, and Jie-Kai Chang. Apache Mahout QDP: GPU-Accelerated Quantum Data Plane. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/apache/mahout.

1 INTRODUCTION

Quantum machine learning (QML) combines quantum computing with classical machine learning to explore potential computational advantages [4, 9]. A critical step in every QML pipeline is *quantum data encoding*: transforming classical feature vectors into quantum states that can be processed by parameterized quantum circuits. Common encoding strategies include amplitude encoding, angle encoding, basis encoding, and IQP-style encoding [9].

In current practice, frameworks such as PennyLane [3] and Qiskit [8] implement encoding by constructing and simulating a quantum circuit for each input vector. Even with GPU-accelerated backends (e.g., Lightning.GPU, Aer GPU), the encoding step remains sequential and per-vector, creating a bottleneck that grows with qubit count. For a 16-qubit system, each state vector has $2^{16} = 65{,}536$ complex amplitudes; encoding thousands of training samples at this scale can take minutes with existing tools, while the subsequent quantum circuit simulation on GPU completes in seconds. This *data starvation* problem, where the GPU sits idle waiting for encoded data, is analogous to the classical ML data loading bottleneck that motivated systems like PyTorch DataLoader [7], but has received almost no attention in the quantum computing community.

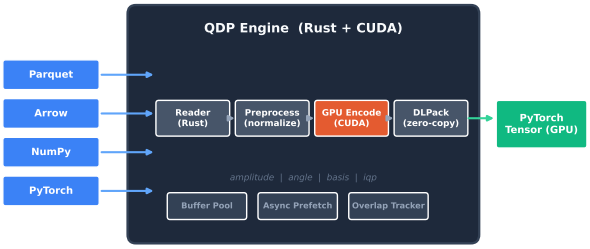


Figure 1: QDP pipeline architecture. Classical data flows from columnar storage through Rust-based readers and preprocessing into CUDA-accelerated encoding kernels, producing GPU-resident quantum state tensors via DLPack.

We present Apache Mahout **QDP** (Quantum Data Plane), a GPU-accelerated data pipeline that treats quantum state encoding as a systems-level data processing problem, analogous to GPU-accelerated ETL in modern data warehouses. QDP addresses this bottleneck through three key design choices:

(1) **Zero-copy data path.** QDP reads directly from Parquet [2] and Arrow [1] via native Rust readers, and exposes results to PyTorch via DLPack [5] with no host-device copy. Data flows from disk to GPU without touching the Python heap.
(2) **Direct computation, no circuit simulation.** Rather than constructing and simulating gate sequences, QDP computes encoded state vectors analytically from classical input data, bypassing circuit simulation entirely.
(3) **Fused GPU kernels with Rust core.** Batched CUDA kernels process all samples in a single launch, while the entire pipeline runs in Rust, releasing the Python GIL so that encoding and training proceed concurrently.

*Equal contribution

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Together, these design choices yield **7–1,000**× speedup in data-to-state latency depending on encoding method, qubit count, and baseline framework. Even against PennyLane's GPU-accelerated `lightning.gpu`, QDP achieves 7–24× lower latency by bypassing circuit simulation entirely. In end-to-end QML training on SVHN, QDP's one-shot encoding reduces training time by 1.26–1.50× while producing identical accuracy. QDP is open source as part of Apache Mahout and is available on PyPI, providing a Python API compatible with PyTorch through zero-copy DLPack tensors.

2 DESIGN AND IMPLEMENTATION

Figure 1 shows the QDP pipeline architecture. The system consists of four stages connected by an asynchronous buffer pool: data ingestion from columnar storage, CPU-side preprocessing, GPU-accelerated encoding, and zero-copy tensor output. Each stage is implemented in Rust for memory safety and GIL-free concurrency, with CUDA kernels handling the encoding step.

2.1 Data Ingestion

QDP ingests data from multiple formats: Parquet, Arrow IPC, NumPy, PyTorch, and TFRecord. For Parquet and Arrow, QDP uses native Rust readers (`arrow-rs`, `parquet` crates) that decode columnar data directly into contiguous memory, avoiding Python deserialization. A streaming mode reads Parquet row groups incrementally for datasets larger than memory. Remote URLs (`s3://`, `gs://`) are supported via the `object_store` crate for cloud data lake ingestion.

Raw feature vectors are then normalized according to the requirements of each encoding method. For amplitude encoding, this involves L2 normalization to ensure $\sum_i |a_i|^2 = 1$. Preprocessing runs on the CPU using multi-threaded Rust code and writes results into pinned host memory for efficient DMA transfer to the GPU.

2.2 Fused GPU Encoding

Existing frameworks implement encoding by simulating a quantum circuit through sequential gate applications to a state vector. However, state preparation is fundamentally a classical computation: encoding maps classical vectors to state amplitudes via well-defined mathematical transformations that can be computed directly and efficiently on GPUs [4, 9]. While prior work has identified this opportunity, QDP is the first system to treat quantum data encoding as a systems-level engineering problem, providing fused CUDA kernels that compute encoded state vectors analytically from classical input data with native batching across multiple samples in one kernel launch. QDP currently supports four widely used encoding methods: *amplitude* (normalize as quantum amplitudes), *angle* (map features to rotation angles), *basis* (encode integer as $|k\rangle$), and *IQP* (entangling interactions between qubits), with additional methods under active development.

Each encoding method is implemented as a fused CUDA kernel that reads classical input and writes the encoded state vector in a single pass. All kernels share a common design that also serves as the template for future encoding methods: vectorized memory access for high bandwidth, on-GPU normalization to avoid CPU preprocessing, and launch on dedicated CUDA streams to overlap encoding with data transfers. Encoded state vectors remain in GPU memory and are exposed to PyTorch via DLPack [5] zero-copy interop, requiring no host-device transfer or memory duplication.

2.3 Pipeline Infrastructure

Beyond the encoding kernels, QDP includes pipeline infrastructure to keep the GPU fully utilized throughout the data loading process. Three subsystems work together to achieve this:

Dual-stream pipeline. QDP allocates two CUDA streams: one for host-to-device memory copies and one for compute kernels, synchronized via CUDA events. This double-buffered design ensures that batch i+1 is being transferred while batch i is being encoded, hiding transfer latency behind compute. At high qubit counts (16+), where each state vector is 512 KB–16 MB, overlapping the PCIe transfer with GPU compute is essential to prevent the GPU from stalling.

Pinned buffer pool. Repeated allocation and deallocation of pinned memory is expensive due to kernel-mode transitions. QDP pre-allocates a fixed pool of page-locked host buffers at initialization. Buffers are checked out via RAII handles and automatically returned on drop, amortizing allocation cost across all batches. A condition variable blocks producers when all buffers are in use, providing natural backpressure and bounding memory usage.

Overlap tracker. An instrumentation layer records timestamps for copy and compute events on each batch, computing the fraction of time that both streams are active simultaneously. This metric, exposed through the Python API, helps users identify whether their workload is transfer-bound or compute-bound and tune batch sizes accordingly.

2.4 Programming Interface

QDP exposes two Python APIs. The **one-shot API** encodes an entire file into a single GPU tensor, suitable for small datasets that fit in memory:

```
import qumat.qdp as qdp
engine = qdp.QdpEngine(device_id=0)
qt = engine.encode("data.parquet", num_qubits=16,
                   encoding_method="amplitude")
tensor = torch.from_dlpack(qt) # (N, 65536) complex64
```

The **iterator API** provides DataLoader-style batching with a builder pattern, streaming encoded batches for large-scale training loops:

```
loader = (qdp.QuantumDataLoader(device_id=0)
         .qubits(16).encoding("amplitude")
         .batches(100, size=64)
         .source_file("data.parquet"))
for batch in loader:
    state = torch.from_dlpack(batch) # (64, 65536)
```

The iterator internally drives the dual-stream pipeline (§2.3), releasing the Python GIL during Rust/CUDA execution so that model training can proceed concurrently. Input sources include local files (Parquet, Arrow, NumPy, PyTorch, TFRecord), in-memory arrays, PyTorch tensors, and remote URLs (`s3://`, `gs://`).

GIL release and concurrency. A key bottleneck in Python-based QML frameworks is the Global Interpreter Lock (GIL), which serializes all Python execution. QDP's Rust core releases the GIL before entering the encoding pipeline, allowing the Python thread to run model training, data augmentation, or other tasks while encoding proceeds on the GPU. This is particularly important for training loops where encoding and gradient computation can overlap.

Precision control. QDP supports both `float32` (complex64) and `float64` (complex128) output, selectable at engine initialization via `QdpEngine(precision="float64")`. The default `float32` minimizes GPU memory and bandwidth; `float64` is available when numerical precision is critical, such as for correctness validation against reference implementations.

3 EVALUATION

We evaluate Apache Mahout QDP on an NVIDIA A100 GPU (80 GB) with amplitude, angle, and basis encoding across 2–20 qubits, comparing against PennyLane (`default.qubit` and `lightning.gpu`) and Qiskit. All measurements include GPU transfer time.

Scaling. Figure 2 shows latency scaling across qubit counts for all three encoding methods, comparing four frameworks. QDP consistently achieves the lowest latency across all configurations, even against PennyLane's GPU-accelerated `lightning.gpu` backend. For angle encoding at 20 qubits, QDP (0.27 ms) is 1,018× faster than PennyLane `default.qubit` (274 ms) and 24× faster than `lightning.gpu` (6.5 ms). For basis encoding, the gap is similarly large: QDP (0.26 ms) vs Qiskit (180 ms) at 20 qubits. Qiskit's circuit-based amplitude encoding grows exponentially, reaching 171 ms at just 10 qubits, while QDP remains at 0.11 ms. The advantage stems from QDP's fused kernel approach: where PennyLane and Qiskit apply $O(n)$ to $O(n^2)$ sequential gate operations, QDP computes all 2^n amplitudes in a single parallel kernel launch.

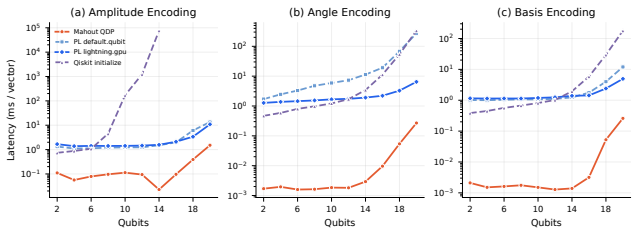


Figure 2: Data-to-state latency scaling (ms/vector, log scale) for amplitude, angle, and basis encoding. QDP (orange) achieves orders-of-magnitude lower latency than both CPU and GPU backends.

Correctness. Since QDP bypasses circuit simulation with fused CUDA kernels, correctness validation is essential. We compare QDP's encoded states against Qiskit across all three encoding methods (amplitude, angle, basis) and 10 qubit counts from 2 to 20, using float64 precision with 50 random input vectors per configuration. Table 1 shows the maximum amplitude difference is below 10^{-16} (machine epsilon) in every case, confirming that QDP produces numerically identical quantum states despite using a fundamentally different computation path.

Case study: SVHN classification. To validate end-to-end impact, we train an IQP-based variational quantum classifier on the SVHN dataset [6] (binary classification: digits 1 vs 7, 1,000 samples, 200 iterations) across 12–20 qubits on A100. QDP performs one-shot encoding (~10 ms for all samples), while PennyLane re-encodes via IQP gates on every forward pass. Both converge to identical test accuracy (0.70). Table 2 shows QDP-GPU achieves 1.26–1.50× training speedup over PennyLane `lightning.gpu`. The speedup comes from *circuit simplification*: PennyLane embeds $O(n^2)$ IQP encoding gates into the circuit and re-executes them on every forward pass as part of circuit simulation, while QDP pre-computes encoded states and removes these gates entirely, allowing the training loop to run a simpler variational-only circuit. Our evaluation is limited to simulators (up to 20 qubits); larger QML workloads may exhibit different bottleneck profiles.

Table 1: Max state-vector error: QDP vs Qiskit (float64).

Qubits	Amplitude	Angle	Basis
12	1.4×10^{-17}	8.3×10^{-17}	0
14	6.9×10^{-18}	4.9×10^{-17}	0
16	6.9×10^{-18}	4.2×10^{-17}	0
18	1.7×10^{-18}	2.1×10^{-17}	0
20	1.7×10^{-18}	1.7×10^{-17}	0

Table 2: End-to-end training time (seconds) on SVHN IQP classification (1,000 samples, A100). Both achieve identical accuracy (0.70).

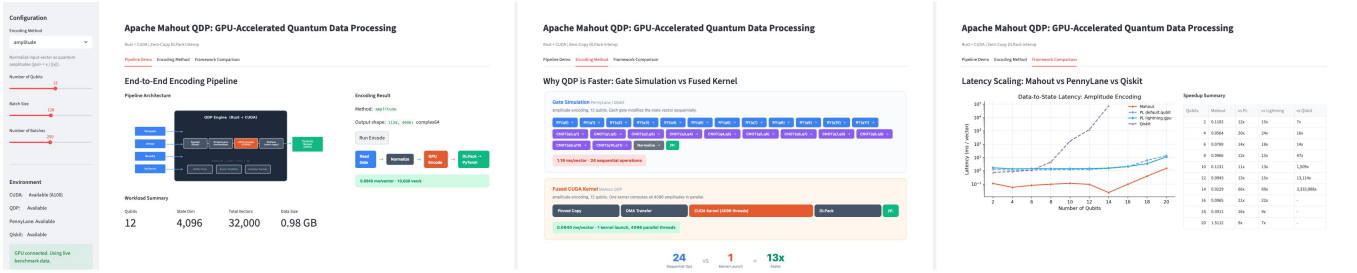
Qubits	PL-GPU	QDP-GPU	Speedup
12	159.5	106.0	1.50×
14	193.4	146.1	1.32×
16	220.6	147.5	1.49×
18	250.2	199.2	1.26×
20	354.3	243.5	1.45×

4 DEMONSTRATION

We provide an interactive web-based demonstration built with Streamlit that showcases QDP's capabilities through three scenarios. The demonstration runs on a machine equipped with an NVIDIA GPU and allows attendees to interact with the system in real time.

4.1 Scenario 1: Pipeline Demo

In the first scenario (Figure 3a), attendees explore the end-to-end encoding pipeline. A sidebar provides global configuration: encoding method (amplitude, angle, basis, IQP), qubit count (2–20), batch size (1–256), and number of batches (10–500). The main panel displays the pipeline architecture diagram alongside a workload summary showing the state vector dimension (2^n), total number of vectors, and estimated data size in GB. Clicking "Run Encode" triggers a real-time encoding pass through the QDP engine. An animated pipeline visualization appears below the button, sequentially highlighting each stage (Read Data, Normalize, GPU Encode, DLPack) to illustrate the data flow. The measured per-vector latency and aggregate throughput are displayed upon completion. Attendees can freely adjust parameters and re-run, observing how encoding cost changes with qubit count, encoding method, and batch configuration.



(a) Pipeline Demo (b) Encoding Method (c) Framework Comparison

Figure 3: QDP interactive demonstration. (a) Pipeline demo with live encoding and animated pipeline stages. (b) Encoding method comparison: gate simulation (sequential) vs fused CUDA kernel (parallel). (c) Framework comparison with latency scaling across 2–20 qubits for four frameworks.

4.2 Scenario 2: Encoding Method

The second scenario (Figure 3b) visually explains *why* QDP is faster than existing frameworks. Two side-by-side cards contrast the gate simulation approach used by PennyLane and Qiskit with QDP's fused CUDA kernel. The upper card (Gate Simulation) displays each quantum gate as a colored pill: RY gates for rotation, CNOT gates for entanglement, and a final normalization step. Gates appear sequentially via CSS animation, emphasizing the per-gate overhead. The lower card (Fused CUDA Kernel) shows QDP's four-stage pipeline (pinned copy, DMA transfer, CUDA kernel, DLPack), where a single kernel launch computes all 2^n amplitudes in parallel. A summary bar at the bottom highlights the contrast: for 12-qubit amplitude encoding, gate simulation requires 24 sequential operations while QDP uses a single kernel launch, achieving 13× speedup. Changing the qubit count or encoding method in the sidebar dynamically updates the gate list, operation count, and speedup ratio, allowing attendees to explore how circuit complexity grows with qubit count.

4.3 Scenario 3: Framework Comparison

The third scenario (Figure 3c) presents quantitative performance data across Mahout QDP, PennyLane (both CPU and GPU backends), and Qiskit. A latency scaling chart plots per-vector encoding time on a log scale across 2–20 qubits, with a speedup table alongside. Switching the encoding method in the sidebar instantly updates both views. QDP's advantage is consistent across all methods, ranging from 7–24× over `lightning.gpu` to over 1,000× over `default.qubit`, depending on encoding and qubit count.

5 CONCLUSION

We demonstrated Apache Mahout QDP, a GPU-accelerated quantum data pipeline that addresses the data loading bottleneck in QML pipelines. By implementing the full path from columnar storage to GPU-resident quantum states in Rust and CUDA, QDP achieves 7–1,000× speedup over CPU-based backends and 7–24× over PennyLane's GPU-accelerated `lightning.gpu`, depending on encoding method and qubit count. Beyond raw encoding speed, QDP's one-shot encoding decouples state preparation from circuit execution, eliminating per-iteration gate overhead and improving end-to-end training time by 1.26–1.50×. Key engineering contributions include dual-stream pipelining, a pinned buffer pool with RAII-based lifecycle management, and zero-copy DLPack interop with PyTorch. Future directions include multi-GPU distribution, AMD ROCm support, and integration with quantum hardware backends.

ACKNOWLEDGMENTS

The authors initiated the QDP project within Apache Mahout and serve as PMC members and committers. We thank Chia-Ping Tsai, Andrew Musselman, and Trevor Grant for their mentorship and guidance on the Apache Mahout project. We also thank the Mahout community and the Apache Taipei[1] community for their support.

REFERENCES

[1] Apache Arrow contributors. 2024. Apache Arrow: A Cross-Language Development Platform for In-Memory Analytics. https://arrow.apache.org.
[2] Apache Parquet contributors. 2024. Apache Parquet: An Open Columnar Storage Format. https://parquet.apache.org.
[3] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, et al. 2022. PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations. In *arXiv:1811.04968*.
[4] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. 2017. Quantum Machine Learning. *Nature* 549 (2017), 195–202.
[5] DLPack contributors. 2023. DLPack: Open In Memory Tensor Structure. https://github.com/dmlc/dlpack.
[6] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. 2011. Reading Digits in Natural Images with Unsupervised Feature Learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*.
[7] Adam Paszke, Sam Gross, Francisco Massa, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[8] Qiskit contributors. 2024. Qiskit: An Open-source Framework for Quantum Computing. (2024). https://doi.org/10.5281/zenodo.2573505
[9] Maria Schuld and Francesco Petruccione. 2021. Machine Learning with Quantum Computers. *Quantum Science and Technology* (2021).

[1] https://github.com/opensource4you